



Lab Sheet 5

Database Management System Lab (BTCS203P)

B. TECH AI&ML, CYBER SECURITY, Data Science 3RD Semester (Session 2026-2025)

Objective:

To understand and practice SQL Joins.

Prerequisites:

1. Basic knowledge of SQL.
2. MySQL installed and set up.

Lab Sheet 5: SQL Joins

Use suitable tables such as Student, Department, Course, and Employee to perform the following:

```
CREATE DATABASE DBMS_Lab5;  
USE DBMS_Lab5;
```

```
-- =====  
-- DEPARTMENT TABLE  
-- =====
```

```
CREATE TABLE Department (  
    Department_ID INT PRIMARY KEY,  
    Department_Name VARCHAR(50)  
);
```

```
INSERT INTO Department VALUES  
(1, 'Computer Science'),  
(2, 'Data Science'),  
(3, 'Artificial Intelligence'),  
(4, 'Cyber Security'),  
(5, 'Electronics');
```

```
-- =====  
-- COURSE TABLE  
-- =====
```

```
CREATE TABLE Course (  
    Course_ID INT PRIMARY KEY,  
    Course_Name VARCHAR(50),  
    Department_ID INT
```

```
);
```

```
INSERT INTO Course VALUES
(101, 'Data Analytics', 2),
(102, 'Machine Learning', 3),
(103, 'Database Management System', 1),
(104, 'Cyber Security Fundamentals', 4),
(105, 'Web Development', 1),
(106, 'Cloud Computing', 2);
```

```
-- =====
-- STUDENT TABLE
-- =====
```

```
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Student_Name VARCHAR(50),
    Department_ID INT,
    Course_ID INT
);
```

```
INSERT INTO Student VALUES
(1, 'Aarav', 1, 103),
(2, 'Priya', 2, 101),
(3, 'Rahul', 2, 106),
(4, 'Ananya', 3, 102),
(5, 'Riya', 4, 104),
(6, 'Karan', NULL, 105),
(7, 'Simran', 1, 105),
(8, 'Vikas', NULL, NULL);
```

```
-- =====
-- EMPLOYEE TABLE
-- =====
```

```
CREATE TABLE Employee (
    Employee_ID INT PRIMARY KEY,
    Employee_Name VARCHAR(50),
    Department_ID INT,
    Salary DECIMAL(10,2),
    Manager_ID INT
);
```

```
INSERT INTO Employee VALUES
(201, 'Rajesh', 1, 80000, NULL),
(202, 'Amit', 1, 55000, 201),
(203, 'Neha', 2, 60000, 201),
(204, 'Rohan', 2, 72000, 201),
(205, 'Sneha', 3, 65000, 201),
(206, 'Pooja', 4, 50000, 201),
(207, 'Vivek', 3, 90000, 201);
```

```
-- Check tables
```

```

SELECT * FROM Student;
SELECT * FROM Department;
SELECT * FROM Course;
SELECT * FROM Employee;

```

Result Grid				
		Filter Rows:		
	Student_ID	Student_Name	Department_ID	Course_ID
▶	1	Aarav	1	103
	2	Priya	2	101
	3	Rahul	2	106
	4	Ananya	3	102
	5	Riya	4	104
	6	Karan	NULL	105
	7	Simran	1	105
	8	Vikas	NULL	NULL
•	NULL	NULL	NULL	NULL

Result Grid		
		Filter Rows:
	Department_ID	Department_Name
▶	1	Computer Science
	2	Data Science
	3	Artificial Intelligence
	4	Cyber Security
	5	Electronics
•	NULL	NULL

Result Grid			
		Filter Rows:	
	Course_ID	Course_Name	Department_ID
▶	101	Data Analytics	2
	102	Machine Learning	3
	103	Database Management System	1
	104	Cyber Security Fundamentals	4
	105	Web Development	1
	106	Cloud Computing	2
•	NULL	NULL	NULL

Result Grid					
		Filter Rows:			
	Employee_ID	Employee_Name	Department_ID	Salary	Manager_ID
▶	201	Rajesh	1	80000.00	NULL
	202	Amit	1	55000.00	201
	203	Neha	2	60000.00	201
	204	Rohan	2	72000.00	201
	205	Sneha	3	65000.00	201
	206	Pooja	4	50000.00	201
	207	Vivek	3	90000.00	201
•	NULL	NULL	NULL	NULL	NULL

1. Demonstrate an INNER JOIN between Student and Department.
SELECT

```

Student.Student_ID,
Student.Student_Name,
Department.Department_Name
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID;

```

Result Grid			
Filter Rows:			
	Student_ID	Student_Name	Department_Name
▶	1	Aarav	Computer Science
	2	Priya	Data Science
	3	Rahul	Data Science
	4	Ananya	Artificial Intelligence
	5	Riya	Cyber Security
	7	Simran	Computer Science

2. Display student names along with their department names using INNER JOIN.

```

SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID;

```

Result Grid		
Filter Rows:		
	Student_Name	Department_Name
▶	Aarav	Computer Science
	Priya	Data Science
	Rahul	Data Science
	Ananya	Artificial Intelligence
	Riya	Cyber Security
	Simran	Computer Science

3. Display employee names along with their department names.

```

SELECT
    Employee.Employee_Name,
    Department.Department_Name
FROM Employee
INNER JOIN Department
ON Employee.Department_ID = Department.Department_ID;

```

Result Grid			Filter Rows:
	Employee_Name	Department_Name	
▶	Rajesh	Computer Science	
	Amit	Computer Science	
	Neha	Data Science	
	Rohan	Data Science	
	Sneha	Artificial Intelligence	
	Pooja	Cyber Security	
	Vivek	Artificial Intelligence	

4. Display students along with their course details.

```
SELECT
    Student.Student_Name,
    Course.Course_Name
FROM Student
INNER JOIN Course
ON Student.Course_ID = Course.Course_ID;
```

	Student_Name	Course_Name
▶	Aarav	Database Management System
	Priya	Data Analytics
	Rahul	Cloud Computing
	Ananya	Machine Learning
	Riya	Cyber Security Fundamentals
	Karan	Web Development
	Simran	Web Development

5. Find students belonging to a particular department using INNER JOIN.

```
SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID
WHERE Department.Department_Name = 'Data Science';
```

Result Grid			Filter Rows:
	Student_Name	Department_Name	
▶	Priya	Data Science	
	Rahul	Data Science	

6. Demonstrate a LEFT OUTER JOIN between Student and Department.

```
SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
LEFT JOIN Department
ON Student.Department_ID = Department.Department_ID;
```

Result Grid			Filter Rows:
	Student_Name	Department_Name	
▶	Aarav	Computer Science	
	Priya	Data Science	
	Rahul	Data Science	
	Ananya	Artificial Intelligence	
	Riya	Cyber Security	
	Karan	NULL	
	Simran	Computer Science	
	Vikas	NULL	

7. Display all students, including those who do not have a matching department.

```
SELECT
    Student.Student_ID,
    Student.Student_Name,
    Department.Department_Name
FROM Student
LEFT JOIN Department
ON Student.Department_ID = Department.Department_ID;
```

Result Grid				Filter Rows:
	Student_ID	Student_Name	Department_Name	
▶	1	Aarav	Computer Science	
	2	Priya	Data Science	
	3	Rahul	Data Science	
	4	Ananya	Artificial Intelligence	
	5	Riya	Cyber Security	
	6	Karan	NULL	
	7	Simran	Computer Science	
	8	Vikas	NULL	

8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

```
SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
RIGHT JOIN Department
ON Student.Department_ID = Department.Department_ID;
```

Result Grid			Filter Rows:
	Student_Name	Department_Name	
▶	Simran	Computer Science	
	Aarav	Computer Science	
	Rahul	Data Science	
	Priya	Data Science	
	Ananya	Artificial Intelligence	
	Riya	Cyber Security	
	HULL	Electronics	

9. Display all departments, including departments having no students.

```
SELECT
    Department.Department_Name,
    Student.Student_Name
FROM Student
RIGHT JOIN Department
ON Student.Department_ID = Department.Department_ID;
```

Result Grid			Filter Rows:
	Department_Name	Student_Name	
▶	Computer Science	Simran	
	Computer Science	Aarav	
	Data Science	Rahul	
	Data Science	Priya	
	Artificial Intelligence	Ananya	
	Cyber Security	Riya	
	Electronics	HULL	

10. Find students who are not assigned to any department using LEFT JOIN.

```
SELECT
    Student.Student_ID,
    Student.Student_Name
FROM Student
LEFT JOIN Department
ON Student.Department_ID = Department.Department_ID
WHERE Department.Department_ID IS NULL;
```

Result Grid			Filter Rows:
	Student_ID	Student_Name	
▶	6	Karan	
	8	Vikas	

11. Find departments having no students using RIGHT/LEFT JOIN.

```
SELECT
    Department.Department_ID,
    Department.Department_Name
```

```

FROM Department
LEFT JOIN Student
ON Department.Department_ID = Student.Department_ID
WHERE Student.Student_ID IS NULL;

```

Result Grid			Filter Rows:
	Department_ID	Department_Name	
▶	5	Electronics	

12. Demonstrate a FULL OUTER JOIN concept using suitable queries/workaround where supported.

```

SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
LEFT JOIN Department
ON Student.Department_ID = Department.Department_ID

UNION

SELECT
    Student.Student_Name,
    Department.Department_Name
FROM Student
RIGHT JOIN Department
ON Student.Department_ID = Department.Department_ID;

```

Result Grid			Filter Rows:
	Student_Name	Department_Name	
▶	Aarav	Computer Science	
	Priya	Data Science	
	Rahul	Data Science	
	Ananya	Artificial Intelligence	
	Riya	Cyber Security	
	Karan	NULL	
	Simran	Computer Science	
	Vikas	NULL	
	NULL	Electronics	

13. Demonstrate a CROSS JOIN between Student and Course.

```

SELECT
    Student.Student_Name,
    Course.Course_Name
FROM Student
CROSS JOIN Course;

```

Result Grid			Filter Rows:
	Student_Name	Course_Name	
▶	Aarav	Cloud Computing	
	Aarav	Web Development	
	Aarav	Cyber Security Fundamentals	
	Aarav	Database Management System	
	Aarav	Machine Learning	
	Aarav	Data Analytics	
	Priya	Cloud Computing	
	Priya	Web Development	
	Priya	Cyber Security Fundamentals	
	Priya	Database Management System	
	Priya	Machine Learning	
	Priya	Data Analytics	
	Rahul	Cloud Computing	
	Rahul	Web Development	
	Rahul	Cyber Security Fundamentals	
	Rahul	Database Management System	
	Rahul	Machine Learning	
	Rahul	Data Analytics	
	Ananya	Cloud Computing	
	Ananya	Web Development	
	Ananya	Cyber Security Fundamentals	
	Ananya	Database Management System	

Result 21

14. Calculate the number of possible Student-Course combinations using CROSS

Result Grid		Filter Rows
	Total_Combinations	
▶	48	

JOIN.

15. Demonstrate a NATURAL JOIN between two suitable tables.

```
CREATE TABLE Department_Info (
    Department_ID INT,
    Department_Name VARCHAR(50)
);
```

```
INSERT INTO Department_Info VALUES
(1, 'Computer Science'),
```

```

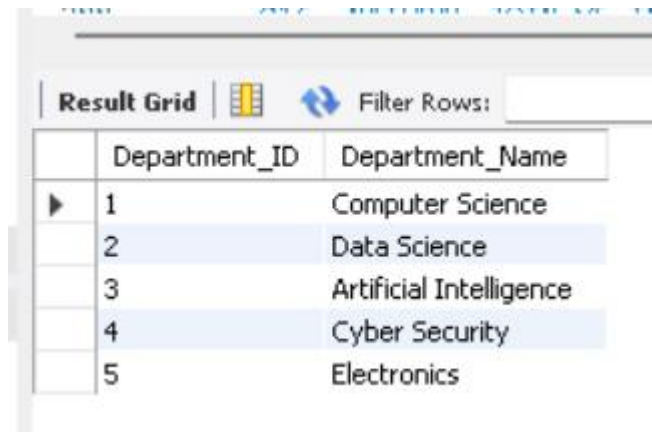
(2, 'Data Science'),
(3, 'Artificial Intelligence'),
(4, 'Cyber Security'),
(5, 'Electronics');

```

```

SELECT *
FROM Department
NATURAL JOIN Department_Info;

```



	Department_ID	Department_Name
▶	1	Computer Science
	2	Data Science
	3	Artificial Intelligence
	4	Cyber Security
	5	Electronics

16. Compare the result of NATURAL JOIN and INNER JOIN.

```

SELECT *
FROM Department
NATURAL JOIN Department_Info;

```

```

SELECT
    Department.Department_ID,
    Department.Department_Name,
    Department_Info.Department_Name AS Info_Department_Name
FROM Department
INNER JOIN Department_Info
ON Department.Department_ID = Department_Info.Department_ID;

```

17. Perform a SELF JOIN on an Employee table to show employee-manager relationships.

```

SELECT
    E.Employee_Name AS Employee,
    M.Employee_Name AS Manager
FROM Employee E
LEFT JOIN Employee M
ON E.Manager_ID = M.Employee_ID;

```



	Employee	Manager
▶	Rajesh	NULL
	Amit	Rajesh
	Neha	Rajesh
	Rohan	Rajesh
	Sneha	Rajesh
	Pooja	Rajesh
	Vivek	Rajesh

18. Display employees along with their managers using SELF JOIN.

```
SELECT
    E.Employee_ID,
    E.Employee_Name AS Employee_Name,
    M.Employee_Name AS Manager_Name
FROM Employee E
LEFT JOIN Employee M
ON E.Manager_ID = M.Employee_ID;
```

Result Grid			
Filter Rows:			
	Employee_ID	Employee_Name	Manager_Name
▶	201	Rajesh	NULL
	202	Amit	Rajesh
	203	Neha	Rajesh
	204	Rohan	Rajesh
	205	Sneha	Rajesh
	206	Pooja	Rajesh
	207	Vivek	Rajesh

19. Join three tables—Student, Department and Course.

```
SELECT
    Student.Student_Name,
    Department.Department_Name,
    Course.Course_Name
FROM Student
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID
INNER JOIN Course
ON Student.Course_ID = Course.Course_ID;
```

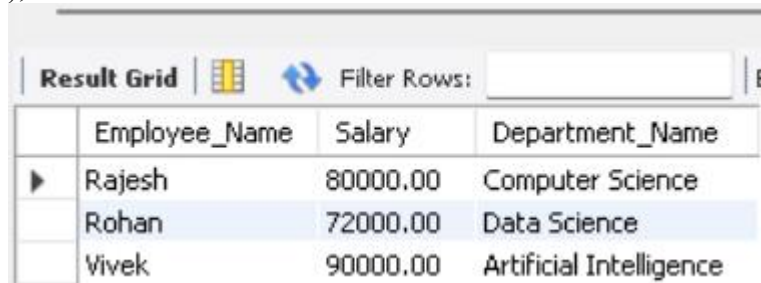
Result Grid			
Filter Rows:			
Export: Wra			
	Student_Name	Department_Name	Course_Name
▶	Aarav	Computer Science	Database Management System
	Priya	Data Science	Data Analytics
	Rahul	Data Science	Cloud Computing
	Ananya	Artificial Intelligence	Machine Learning
	Riya	Cyber Security	Cyber Security Fundamentals
	Simran	Computer Science	Web Development

20. Display student name, course name and department name using multiple joins.

```
SELECT
    Student.Student_Name,
    Course.Course_Name,
    Department.Department_Name
FROM Student
INNER JOIN Course
ON Student.Course_ID = Course.Course_ID
INNER JOIN Department
ON Student.Department_ID = Department.Department_ID;
```

21. Find employees whose salary is greater than the average salary of their department using joins/subqueries.

```
SELECT
    E.Employee_Name,
    E.Salary,
    D.Department_Name
FROM Employee E
INNER JOIN Department D
ON E.Department_ID = D.Department_ID
WHERE E.Salary > (
    SELECT AVG(E2.Salary)
    FROM Employee E2
    WHERE E2.Department_ID = E.Department_ID
);
```

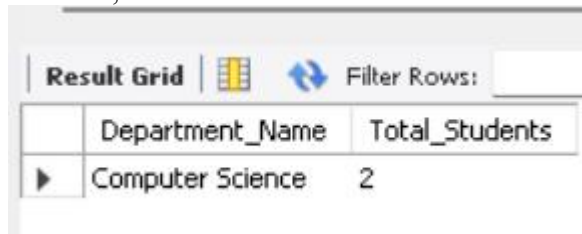


The screenshot shows a 'Result Grid' with a 'Filter Rows' input field. The grid contains three rows of data:

	Employee_Name	Salary	Department_Name
▶	Rajesh	80000.00	Computer Science
	Rohan	72000.00	Data Science
	Vivek	90000.00	Artificial Intelligence

22. Find the department having the maximum number of students using joins and aggregation.

```
SELECT
    D.Department_Name,
    COUNT(S.Student_ID) AS Total_Students
FROM Department D
LEFT JOIN Student S
ON D.Department_ID = S.Department_ID
GROUP BY D.Department_ID, D.Department_Name
ORDER BY Total_Students DESC
LIMIT 1;
```



The screenshot shows a 'Result Grid' with a 'Filter Rows' input field. The grid contains one row of data:

	Department_Name	Total_Students
▶	Computer Science	2

23. Display departments and the total number of students in each department.

```
SELECT
    D.Department_Name,
    COUNT(S.Student_ID) AS Total_Students
FROM Department D
LEFT JOIN Student S
ON D.Department_ID = S.Department_ID
GROUP BY D.Department_ID, D.Department_Name;
```

Result Grid			Filter Rows:
	Department_Name	Total_Students	
▶	Computer Science	2	
	Data Science	2	
	Artificial Intelligence	1	
	Cyber Security	1	
	Electronics	0	

24. Display courses along with the number of students enrolled in each course.

```
SELECT
    C.Course_Name,
    COUNT(S.Student_ID) AS Total_Students
FROM Course C
LEFT JOIN Student S
ON C.Course_ID = S.Course_ID
GROUP BY C.Course_ID, C.Course_Name;
```

Result Grid			Filter Rows:
	Course_Name	Total_Students	
▶	Data Analytics	1	
	Machine Learning	1	
	Database Management System	1	
	Cyber Security Fundamentals	1	
	Web Development	2	
	Cloud Computing	1	

25. Create a complete query using INNER JOIN, LEFT JOIN, GROUP BY and

aggregate functions to generate a student-department-course report.

```
SELECT
    D.Department_Name,
    C.Course_Name,
    COUNT(S.Student_ID) AS Total_Students
FROM Department D
LEFT JOIN Student S
ON D.Department_ID = S.Department_ID
LEFT JOIN Course C
ON S.Course_ID = C.Course_ID
GROUP BY
    D.Department_ID,
    D.Department_Name,
    C.Course_ID,
    C.Course_Name
ORDER BY
    D.Department_Name,
    C.Course_Name;
```

Result Grid			
		Filter Rows:	Export:  Wrap
	Department_Name	Course_Name	Total_Students
▶	Artificial Intelligence	Machine Learning	1
	Computer Science	Database Management System	1
	Computer Science	Web Development	1
	Cyber Security	Cyber Security Fundamentals	1
	Data Science	Cloud Computing	1
	Data Science	Data Analytics	1

Submission:

Take screenshots of each step's output.

Save the SQL scripts in a text file.